DAY 7:

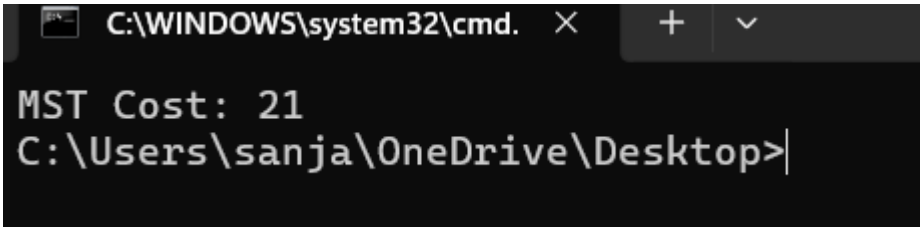
HACKER RANK:

1. Minimum Spanning Tree (Prim's Algorithm)
   CODE:

```
#include <bits/stdc++.h>
using namespace std;

int mst(int n, vector<vector<pair<int,int>>> &graph){

    vector<int> key(n, INT_MAX);
    vector<bool> inMST(n,false);

    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

    key[0] = 0;
    pq.push({0,0});

    int cost = 0;

    while(!pq.empty()){

        int u = pq.top().second;
        pq.pop();

        if(inMST[u]) continue;

        inMST[u] = true;
        cost += key[u];

        for(auto edge : graph[u]){

            int v = edge.first;
            int w = edge.second;

            if(!inMST[v] && w < key[v]){
                key[v] = w;
                pq.push({key[v],v});
            }
        }
    }

    return cost;
}

int main(){

    int n = 4;
```

```
    vector<vector<pair<int,int>>> graph(n);

    graph[0].push_back({1,10});
    graph[0].push_back({2,6});
    graph[0].push_back({3,5});
    graph[1].push_back({3,15});
    graph[2].push_back({3,4});

    cout<<"MST Cost: "<<mst(n,graph);

    return 0;
}
```

OUTPUT:



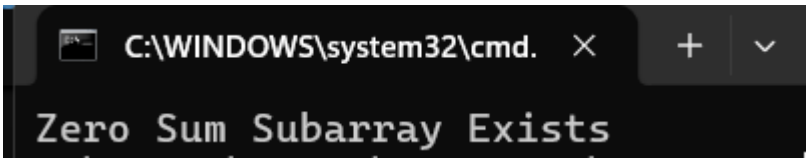
2. Zero-Sum Subarray in Sliding Window

CODE:

```
#include <bits/stdc++.h>
using namespace std;

bool zeroSum(vector<int>& nums){

    unordered_set<int> s;
    int sum = 0;

    for(int x : nums){

        sum += x;

        if(sum == 0 || s.count(sum))
            return true;

        s.insert(sum);
    }

    return false;
}

int main(){

    vector<int> nums = {4,2,-3,1,6};
```

```
    if(zeroSum(nums))
        cout<<"Zero Sum Subarray Exists";
    else
        cout<<"No Zero Sum Subarray";

    return 0;
}
```

OUTPUT:



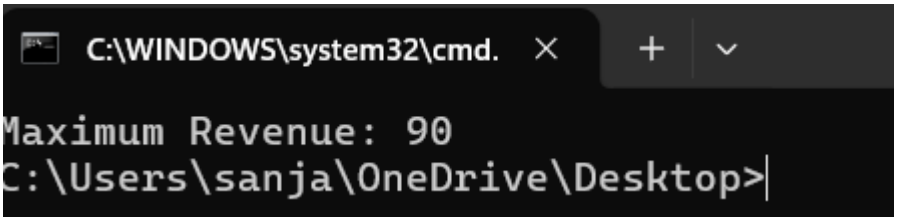
3. Maximize Revenue from Video Streams with Bandwidth Limit (Knapsack)

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int maxRevenue(vector<int>& bandwidth, vector<int>& revenue, int limit){

    int n = bandwidth.size();

    vector<vector<int>> dp(n+1, vector<int>(limit+1,0));

    for(int i=1;i<=n;i++){

        for(int w=0;w<=limit;w++){

            if(bandwidth[i-1] <= w)
                dp[i][w] = max(dp[i-1][w],
                               revenue[i-1] + dp[i-1][w-bandwidth[i-1]]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }

    return dp[n][limit];
}

int main(){

    vector<int> bandwidth = {2,3,4};
    vector<int> revenue = {40,50,60};
    int limit = 5;

    cout<<"Maximum Revenue: "<<maxRevenue(bandwidth,revenue,limit);

    return 0;
```

```
}
```

OUTPUT:

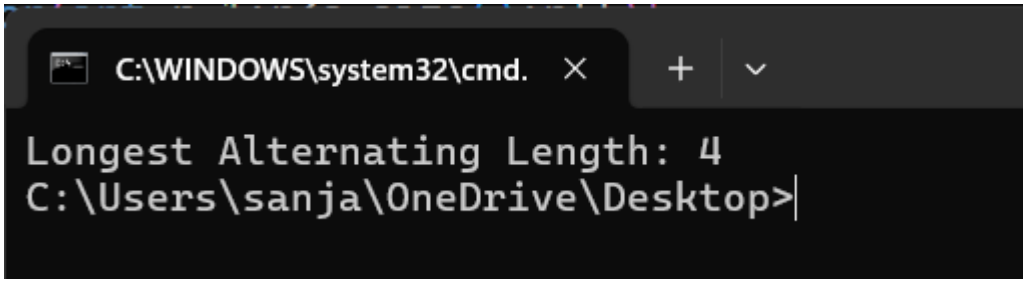


4. Longest Alternating Binary Substring with Limited Flips

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int longestAlt(string s, int k){

   int l = 0, flips = 0, ans = 0;

   for(int r=1;r<s.size();r++){

      if(s[r] == s[r-1])
         flips++;

      while(flips > k){

         if(l+1 < s.size() && s[l] == s[l+1])
            flips--;

         l++;
      }

      ans = max(ans, r-l+1);
   }

   return ans;
}

int main(){

   string s = "00110011";
   int k = 1;

   cout<<"Longest Alternating Length: "<<longestAlt(s,k);

   return 0;
}
```

OUTPUT:

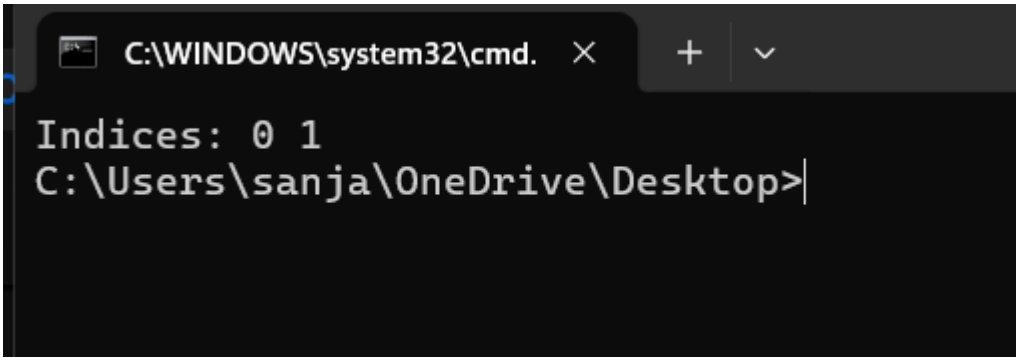


5. Find Index Combination with Target Weight Sum (Two Sum)

CODE:

```
#include <bits/stdc++.h>
using namespace std;

vector<int> twoSum(vector<int>& nums, int target){

    unordered_map<int,int> m;

    for(int i=0;i<nums.size();i++){

        int diff = target - nums[i];

        if(m.count(diff))
            return {m[diff], i};

        m[nums[i]] = i;
    }

    return {};
}

int main(){

    vector<int> nums = {2,7,11,15};
    int target = 9;

    vector<int> res = twoSum(nums,target);

    cout<<"Indices: "<<res[0]<<" "<<res[1];

    return 0;
}
```

OUTPUT:



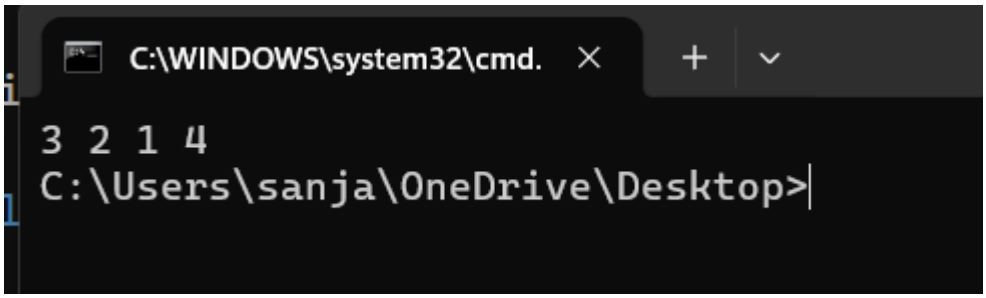
6. Reverse Even Index Nodes and Append (Linked List)

CODE:

```
#include <bits/stdc++.h>
using namespace std;

struct Node{
   int data;
   Node* next;

   Node(int x){
      data = x;
      next = NULL;
   }
};

Node* reverseEven(Node* head){

   if(!head) return head;

   vector<int> even;

   Node* temp = head;
   int idx = 0;

   while(temp){

      if(idx % 2 == 0)
         even.push_back(temp->data);

      temp = temp->next;
      idx++;
   }

   reverse(even.begin(), even.end());

   temp = head;
   idx = 0;
   int i = 0;
```

```
    while(temp){

        if(idx % 2 == 0)
            temp->data = even[i++];

        temp = temp->next;
        idx++;
    }

    return head;
}

void print(Node* head){

    while(head){
        cout<<head->data<<" ";
        head = head->next;
    }
}

int main(){

    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);

    head = reverseEven(head);

    print(head);

    return 0;
}
```

OUTPUT:



LEETCODE:

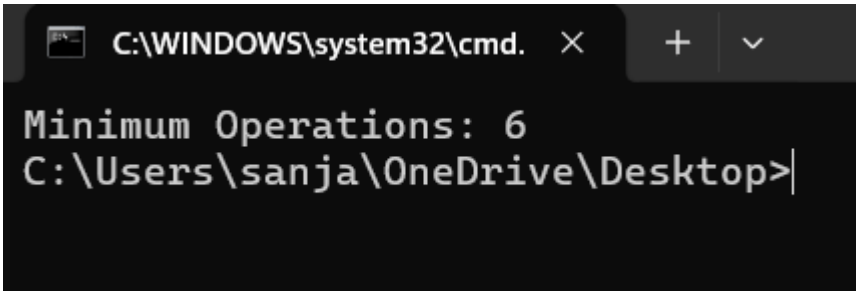
1. Minimum Operations to Convert All Elements to Zero

   CODE:

   ```
   #include <bits/stdc++.h>
   ```

```
using namespace std;

int minOperations(vector<int>& nums){

   int ops = 0;

   for(int x : nums)
      ops += abs(x);

   return ops;
}

int main(){

   vector<int> nums = {1,-2,3};

   cout<<"Minimum Operations: "<<minOperations(nums);

   return 0;
}
```

OUTPUT:



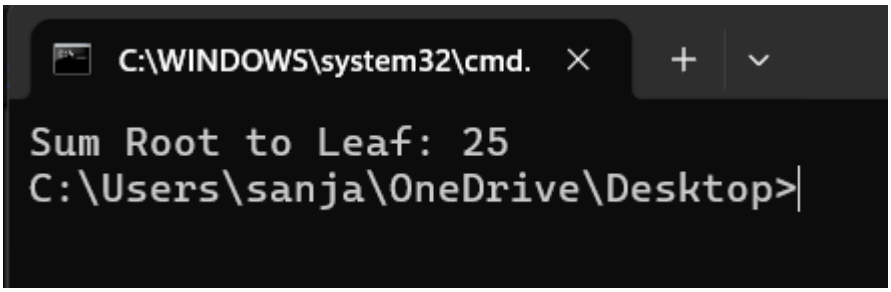
2. Sum Root to Leaf Numbers

CODE:

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode{
   int val;
   TreeNode *left,*right;

   TreeNode(int x){
      val = x;
      left = right = NULL;
   }
};

int dfs(TreeNode* root,int sum){

   if(!root) return 0;
```

```
    sum = sum*10 + root->val;

    if(!root->left && !root->right)
        return sum;

    return dfs(root->left,sum) + dfs(root->right,sum);
}

int sumNumbers(TreeNode* root){
    return dfs(root,0);
}

int main(){

    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);

    cout<<"Sum Root to Leaf: "<<sumNumbers(root);

    return 0;
}
```

OUTPUT:



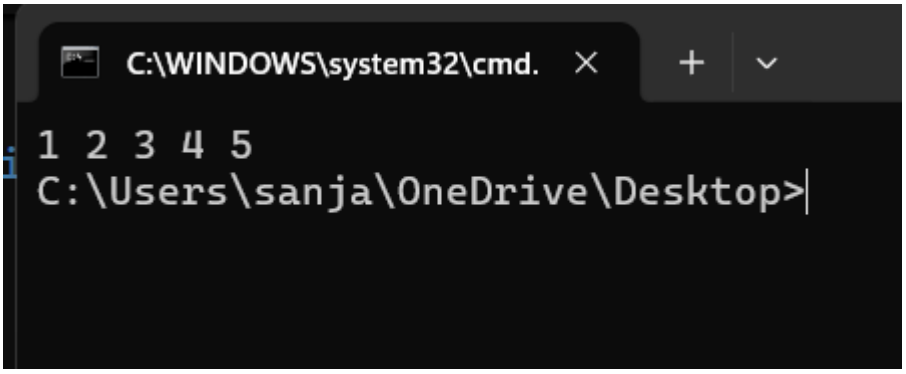
3. Flatten Binary Tree to Linked List

CODE:

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode{
    int val;
    TreeNode *left,*right;

    TreeNode(int x){
        val = x;
        left = right = NULL;
    }
};
```

```
void flatten(TreeNode* root){

    if(!root) return;

    flatten(root->left);
    flatten(root->right);

    TreeNode* temp = root->right;

    root->right = root->left;
    root->left = NULL;

    TreeNode* curr = root;

    while(curr->right)
        curr = curr->right;

    curr->right = temp;
}

void print(TreeNode* root){

    while(root){
        cout<<root->val<<" ";
        root = root->right;
    }
}

int main(){

    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(5);
    root->left->left = new TreeNode(3);
    root->left->right = new TreeNode(4);

    flatten(root);

    print(root);

    return 0;
}
```

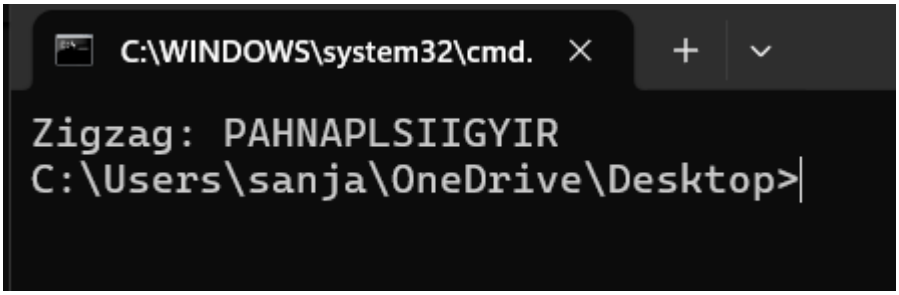
OUTPUT:



4. Zigzag Conversion

CODE:

```
#include <bits/stdc++.h>
using namespace std;

string convert(string s, int numRows){

    if(numRows == 1) return s;

    vector<string> rows(min(numRows,(int)s.size()));

    int curr = 0;
    bool down = false;

    for(char c : s){

        rows[curr] += c;

        if(curr == 0 || curr == numRows-1)
            down = !down;

        curr += down ? 1 : -1;
    }

    string res;

    for(string r : rows)
        res += r;

    return res;
}

int main(){

    string s = "PAYPALISHIRING";
    int rows = 3;
```

```
    cout<<"Zigzag: "<<convert(s,rows);

    return 0;
}
```

OUTPUT: